Assignment 3
Architecture and Design Report

Project: AI-Powered WhatsApp Business Agent (RAG Engine)

Prepared by
Ahmad Bilal Fazal (BSAI24005)
Mah Zarreen Sajjad (BSAI24033)

Table of Contents

1. Introduction

This report presents the architectural design and detailed design for the semester project titled AI-Powered WhatsApp Business Agent (RAG Engine). The project is designed as an intelligent WhatsApp automation system for businesses that need to answer customer questions quickly and accurately. Instead of using only fixed keyword replies, the system allows the administrator to upload business documents and then uses Retrieval-Augmented Generation (RAG) to answer customer questions based on those documents.

The main purpose of the system is to reduce repetitive manual communication for small business owners while improving the quality and speed of responses received by customers. A customer can ask a question in natural language through WhatsApp, and the system retrieves relevant content from the uploaded knowledge base, generates a grounded answer, adds a citation to the source content, and sends the response back through the WhatsApp communication channel.

1.1 Main Features Restated from the SRS

- **Document ingestion:** The administrator can upload PDF, DOCX, and TXT files. The system extracts the text, divides it into smaller chunks, creates embeddings, and stores them in the vector database.
- **Natural language WhatsApp querying:** Customers can send questions in plain English through WhatsApp without needing to use exact keywords.
- **Contextual answer generation:** The RAG pipeline retrieves relevant document chunks and uses a transformer model to generate a response based on the retrieved content.
- **Source citations:** The answer includes the document or page reference used by the system, which improves trust and reduces unsupported responses.
- **Session and chat management:** The administrator can view chat history and analytics through a web-based dashboard.
- **Hallucination control:** If the knowledge base does not contain enough information, the system returns a safe fallback response such as Limited Information Available instead of inventing an answer.

1.2 Technology Stack

Layer / Need	Selected Technology	Purpose in the System
Programming language	Python 3.x	Core implementation language for backend, AI pipeline, file handling, and integration logic.
Backend API / routing	Flask or FastAPI	Receives Twilio webhooks, exposes API endpoints, routes requests to application services.
Admin dashboard	Gradio or Streamlit	Provides a simple web interface for document upload, indexing status, and chat analytics.
Communication bridge	Twilio API for WhatsApp	Connects WhatsApp messages with the backend webhook endpoint.
Development tunnel	ngrok	Allows a local Flask/FastAPI server to receive public webhook requests during development.
Embedding model	Hugging Face sentence-transformers	Converts documents and customer queries into vector embeddings for semantic search.
Generation model	Hugging Face BART/T5 or similar model	Generates natural language answers from retrieved context.
Vector database	FAISS or ChromaDB	Stores document embeddings and supports fast similarity search.
Data storage	SQLite / local JSON / lightweight DB for MVP	Stores chat history, document metadata, session state, and analytics.
Configuration / security	.env environment variables	Stores API keys and sensitive configuration outside source code.

The original project proposal described a locally hosted Flask and Twilio MVP with dynamic keyword mapping. The SRS extends that idea into a more intelligent RAG-based design by adding document ingestion, vector search, and Hugging Face transformer models. Therefore, the architecture below keeps the practical Flask/Twilio/ngrok foundation while upgrading the core response layer into a modular AI pipeline.

2. Architectural Design

2.1 Chosen Architectural Style

The chosen architectural style is a layered, microservice-oriented architecture. The main style is microservice-oriented because the system is divided into clear independent services or logical services: an admin dashboard, a backend webhook/API service, a document ingestion service, a RAG query service, a vector database, and external service adapters for Twilio and Hugging Face. At the same time, the backend follows a layered structure internally: presentation/channel layer, application service layer, AI/retrieval layer, and data layer.

2.1.1 Justification for the Selected Style

This style is suitable for the AI-Powered WhatsApp Business Agent because the system has multiple responsibilities that should not be mixed in one large script. The WhatsApp webhook must be fast and reliable, the document ingestion process may involve heavier file parsing and embedding generation, and the RAG pipeline needs separate retrieval and generation logic. Separating these responsibilities makes the system easier to test, maintain, and extend.

- **Clear separation of concerns:** The dashboard manages administrator actions, the webhook receives customer messages, the ingestion module builds the knowledge base, and the RAG module produces grounded answers.
- **Better maintainability:** A developer can modify the vector database, update the model, or change the WhatsApp provider without rewriting the whole system.
- **AI workflow suitability:** RAG systems naturally have multiple stages: ingest documents, embed text, retrieve relevant chunks, generate answers, and format citations. These stages map well to separate services.
- **MVP-to-SaaS growth path:** During development, the components can run locally using ngrok. Later, the same logical modules can be containerized and deployed independently.
- **Practical balance:** A fully distributed microservices system may be too complex for a semester project, so the design uses service boundaries logically while allowing simple local execution for the MVP.

2.2 Architectural Overview

The system can be understood as three broad layers. The first layer is the presentation and communication layer, which includes the administrator dashboard and WhatsApp/Twilio channel. The second layer is the application service layer, which coordinates document ingestion, message handling, chat sessions, and RAG processing. The third layer is the data and AI layer, which includes the document store, vector database, chat history storage, and transformer models.

Figure 1: High-Level Architecture of the AI-Powered WhatsApp Business Agent

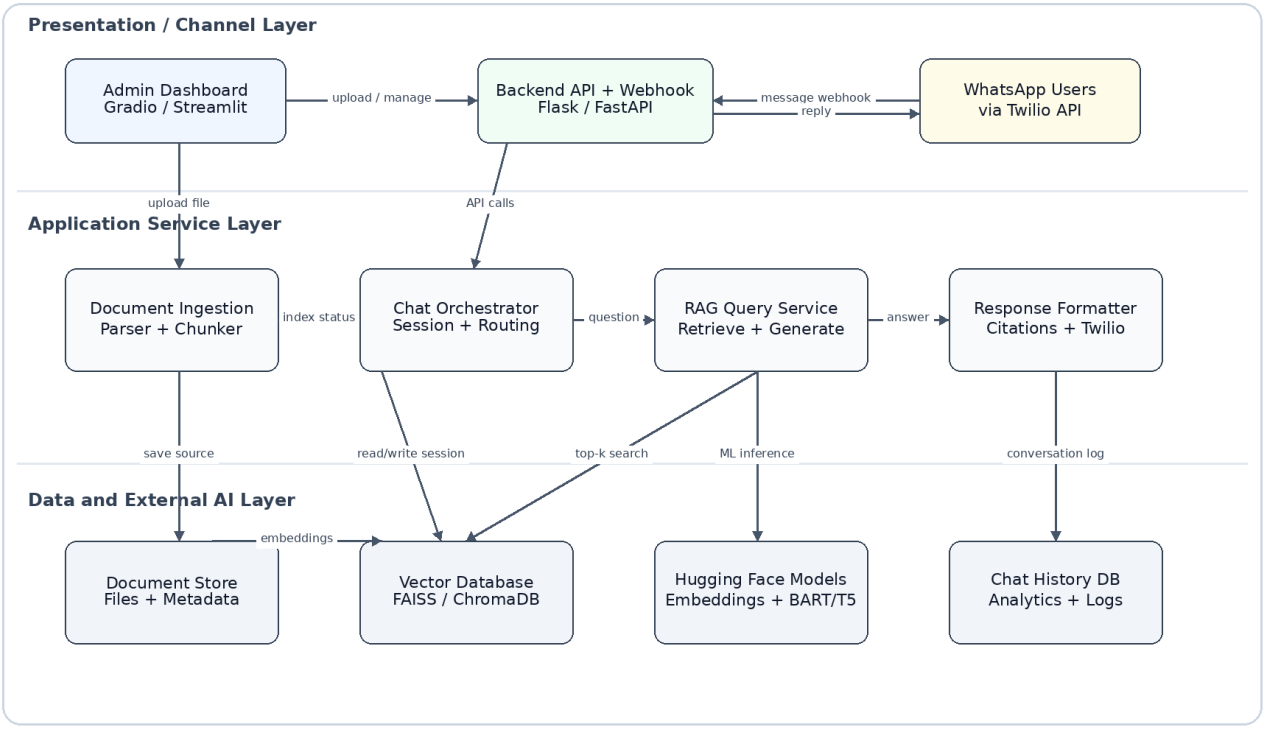


Figure 1: High-Level Architecture of the AI-Powered WhatsApp Business Agent

2.2.1 Major Components and Responsibilities

Component / Module	Main Responsibilities	Technology Mapping
Admin Dashboard	Allows administrator to upload documents, start indexing, view chat history, and review basic analytics.	Gradio or Streamlit frontend.
WhatsApp / Twilio Adapter	Receives user messages from WhatsApp through Twilio and sends generated responses back to the customer.	Twilio WhatsApp API and webhook configuration.
Backend API and Webhook Layer	Provides HTTP endpoints, validates requests, parses incoming payloads, routes requests to the correct service, and returns responses.	Flask or FastAPI application running locally or on a server.
Document Ingestion Service	Validates uploaded files, extracts text, cleans text, chunks content, generates embeddings, and stores vectors and metadata.	Python file parsers, sentence-transformers, FAISS/ChromaDB.
Chat Orchestrator	Controls the customer message flow, manages sessions, stores chat history, calls the RAG service, and handles fallback cases.	Python service class inside backend layer; lightweight DB for session/history.
RAG Query Service	Embeds the customer query, retrieves top relevant chunks, builds grounded prompt, calls generation model, and returns answer with citations.	Hugging Face embedding and generation models; FAISS/ChromaDB.
Vector Database	Stores embeddings and supports semantic similarity search over uploaded document chunks.	FAISS or ChromaDB.
Document and Metadata Store	Stores original uploaded documents, document ids, page numbers, chunk ids, upload time, and indexing status.	Local file system plus SQLite/JSON metadata store for MVP.

Chat History and Analytics Store	Stores customer messages, AI responses, timestamps, citations, session ids, and performance metrics.	SQLite or other lightweight database.
Configuration and Security Layer	Stores secrets, API keys, webhook configuration, and runtime settings safely outside the source code.	.env file and environment variables.

2.2.2 Component Interaction: Document Upload Flow

When the administrator uploads a business document, the dashboard forwards the file to the backend. The document ingestion service validates the file type, extracts text, divides it into meaningful chunks, creates embeddings, and stores both the vector data and the source metadata. After this flow finishes, the document becomes available as part of the business knowledge base.

Figure 2: Document Ingestion and Indexing Flow

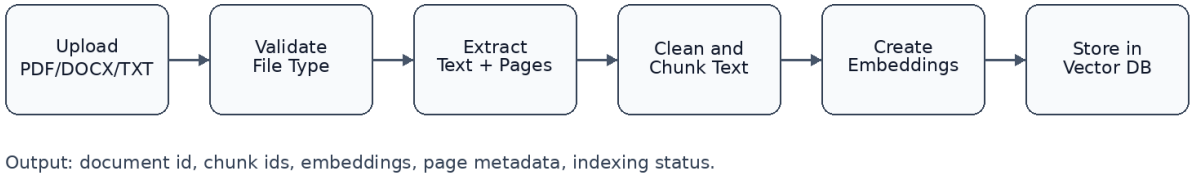


Figure 2: Document Ingestion and Indexing Flow

2.2.3 Component Interaction: WhatsApp Question Flow

When a customer sends a WhatsApp question, Twilio forwards the message to the public webhook URL. During local development, ngrok exposes the Flask/FastAPI server to the internet. The webhook layer extracts the customer number and message body, normalizes the text, calls the RAG query service, and formats the answer. The final response is sent back to the customer through Twilio.

Figure 3: WhatsApp Query and RAG Answer Sequence

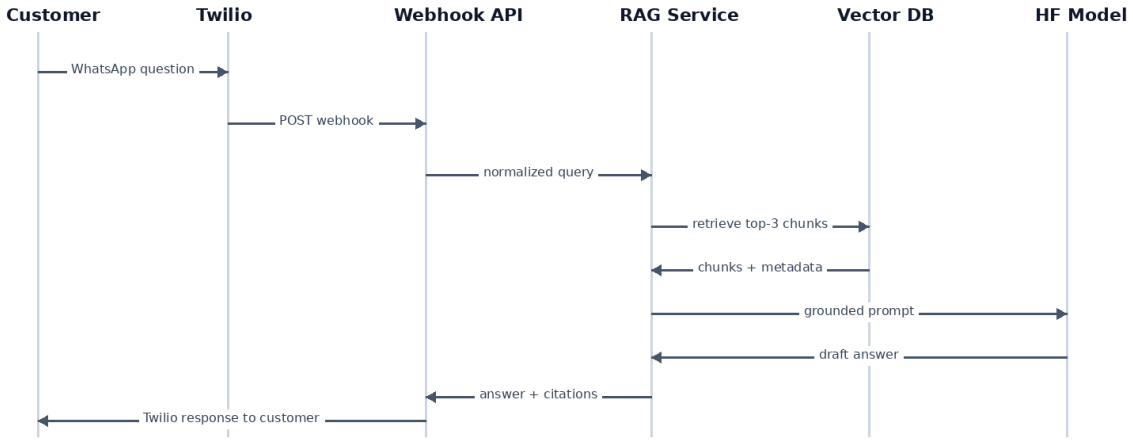


Figure 3: WhatsApp Query and RAG Answer Sequence

2.2.4 Requirement-to-Module Traceability

Requirement	Responsible Module(s)	Design Support
F-01 Document Input Handling	Admin Dashboard, Document Ingestion Service, Vector Database	The dashboard accepts files; ingestion extracts, chunks, embeds, and stores content with metadata.
F-02 WhatsApp Query Input	Twilio Adapter, Backend Webhook, Chat Orchestrator	Twilio sends a webhook request; the backend parses and validates the incoming text message.
F-03 Answer Generation	RAG Query Service, Vector Database, Hugging Face Model	The query is embedded, top chunks are retrieved, and a grounded answer is generated.
Source Citations	RAG Query Service, Metadata Store, Response Formatter	Each retrieved chunk contains document/page metadata that is added to the final answer.
Session Management	Chat Orchestrator, Chat History DB	Messages, responses, timestamps, and session ids are stored for review and analytics.
Performance Target	Webhook Layer, Vector DB, Model Service	Fast vector search and compact prompts support the target WhatsApp response time.
Security Requirement	Configuration Layer, Backend API	Sensitive keys are stored in environment variables; API endpoints can be protected and validated.

3. Detailed Design

This section presents the internal design of three core modules: the Document Ingestion and Knowledge Base Builder, the WhatsApp Webhook and Conversation Orchestration module, and the RAG Answer Generation and Hallucination Control module. These modules were selected because they represent the most important behaviour of the system: building the knowledge base, receiving customer questions, and producing reliable grounded answers.

3.1 Module 1: Document Ingestion and Knowledge Base Builder

3.1.1 Module Description

The Document Ingestion and Knowledge Base Builder module converts uploaded business documents into searchable vector knowledge. Its purpose is to prepare unstructured business information so that the RAG engine can later retrieve the correct context for customer questions. Without this module, the system would only behave like a simple rule-based FAQ bot.

- **Main functionalities:** file validation, text extraction, cleaning, chunking, embedding generation, vector storage, and metadata storage.
- **Inputs:** PDF, DOCX, or TXT file; administrator id; optional business id; upload timestamp.
- **Outputs:** document id, list of chunk ids, vector embeddings, source metadata, indexing status, and error message if the file is invalid.
- **Dependencies:** admin dashboard, file parser, text cleaner, chunker, embedding model, vector database, metadata store, and configuration manager.

3.1.2 Internal Structure

```
+-------------------------------------------------------------+
| DocumentIngestionService                                    |
+-------------------------------------------------------------+
| - validator: FileValidator                                  |
| - extractor: TextExtractor                                  |
| - chunker: TextChunker                                      |
| - embedder: EmbeddingService                                |
| - vector_store: VectorStoreRepository                       |
```

```
| - metadata_store: MetadataRepository                          |
+---------------------------------------------------------------+
| + ingest(file, admin_id) -> IngestionResult                    |
| + rebuild_index(document_id) -> IndexingResult                 |
+---------------------------------------------------------------+

FileValidator        -> checks extension, size, and corrupted file cases
TextExtractor        -> extracts text from PDF, DOCX, TXT
TextChunker          -> splits text into chunks with overlap
EmbeddingService     -> creates vector representations of chunks
VectorStoreRepository-> saves and searches vectors in FAISS/ChromaDB
MetadataRepository   -> saves document name, page no., chunk id, source path
```

3.1.3 Pseudo-code

```
FUNCTION ingest_document(file, admin_id):
    allowed_types = ["pdf", "docx", "txt"]

    IF file.extension NOT IN allowed_types:
        RETURN error("Invalid format or corrupted file")

    TRY:
        document_id = metadata_store.create_document_record(file.name, admin_id)
        raw_pages = text_extractor.extract(file)

        IF raw_pages is empty:
            metadata_store.mark_failed(document_id, "No readable text found")
            RETURN error("Invalid format or corrupted file")

        all_chunks = []
        FOR each page IN raw_pages:
            clean_text = normalize_whitespace(page.text)
            page_chunks = chunker.split(clean_text, chunk_size=500, overlap=80)

            FOR each chunk IN page_chunks:
                chunk_metadata = {
                    "document_id": document_id,
                    "file_name": file.name,
                    "page_number": page.number,
                    "chunk_text": chunk.text
                }
                all_chunks.append((chunk.text, chunk_metadata))

        embeddings = embedding_service.encode([chunk.text for chunk in all_chunks])
        vector_store.upsert(embeddings, [chunk.metadata for chunk in all_chunks])
        metadata_store.mark_indexed(document_id, total_chunks=len(all_chunks))

        RETURN success(document_id, total_chunks=len(all_chunks))

    EXCEPT Exception as e:
        metadata_store.mark_failed(document_id, str(e))
        RETURN error("Invalid format or corrupted file")
```

3.1.4 Design Notes

- Each chunk stores metadata such as document id, filename, page number, and chunk id. This is important because the answer generation module needs these values to produce citations.
- Chunk overlap is used so that an answer is not lost when important information falls between two chunks.

- File validation keeps the knowledge base clean and prevents unsupported files from entering the vector index.
- The module can run synchronously for a small MVP, but it can later be moved to a background worker if large documents are uploaded.

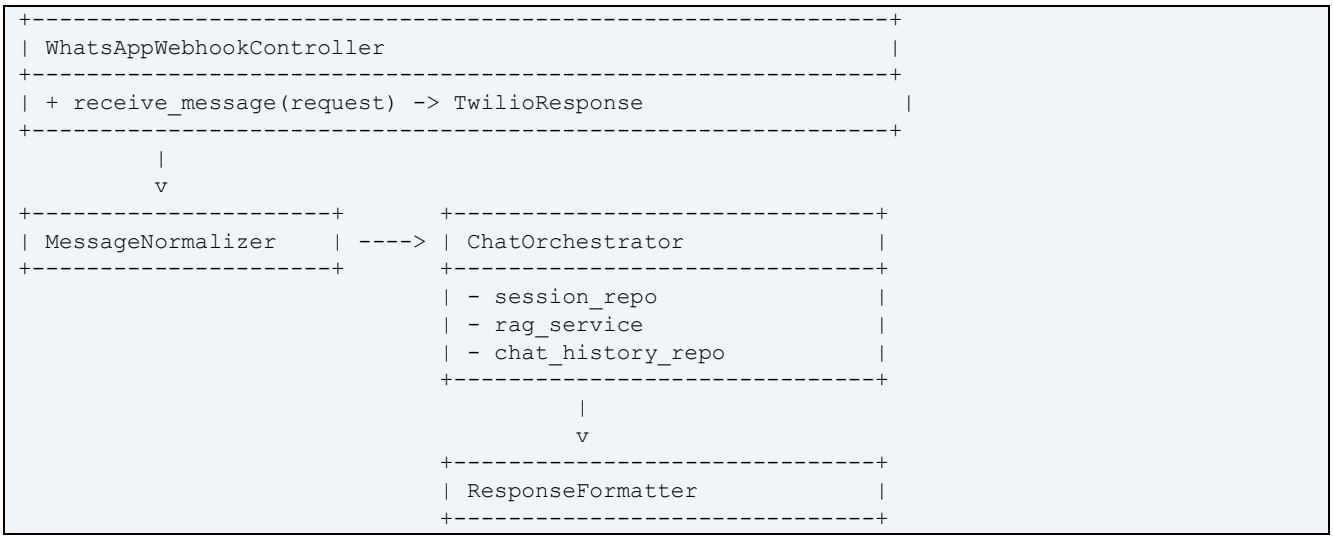
3.2 Module 2: WhatsApp Webhook and Conversation Orchestration

3.2.1 Module Description

The WhatsApp Webhook and Conversation Orchestration module is responsible for handling real-time customer messages. It receives the incoming Twilio webhook request, extracts the WhatsApp sender and message body, normalizes the text, manages the conversation session, calls the RAG answer service, stores the message exchange, and returns the response through Twilio.

- **Main functionalities:** webhook handling, request validation, message parsing, text normalization, session lookup, RAG service call, response formatting, and chat logging.
- **Inputs:** HTTP POST request from Twilio containing fields such as From, Body, MessageSid, and timestamp.
- **Outputs:** AI-generated WhatsApp response, Twilio-compatible response payload, and saved chat history.
- **Dependencies:** Twilio API, ngrok or public server URL, chat/session repository, RAG query service, configuration manager, and logging system.

3.2.2 Internal Structure



3.2.3 Pseudo-code

```
ROUTE POST /webhook/whatsapp:
    sender = request.form["From"]
    user_text = request.form["Body"]
    message_id = request.form.get("MessageSid")

    IF sender is empty OR user_text is empty:
        RETURN twilio_reply("Please send a valid text message.")

    normalized_text = normalizer.clean(user_text)
    session = session_repository.get_or_create(sender)

    start_time = current_time()
```

```
    rag_result = rag_service.answer_question(
        question = normalized_text,
        session_id = session.id
    )
    latency = current_time() - start_time

    IF rag_result.status == "NO_CONTEXT":
        final_text = "Limited Information Available. Please contact support for details."
    ELSE:
        final_text = response_formatter.with_citations(
            answer = rag_result.answer,
            citations = rag_result.citations
        )

    chat_history.save(
        session_id = session.id,
        sender = sender,
        user_message = user_text,
        bot_response = final_text,
        message_id = message_id,
        latency = latency,
        citations = rag_result.citations
    )

    RETURN twilio_reply(final_text)
```

3.2.4 Design Notes

- The webhook controller stays thin. It should only handle HTTP-specific work and then forward business logic to service classes.
- The chat orchestrator provides one place for conversation logic, fallback behaviour, and session state.
- All incoming and outgoing messages are logged. This supports administrator analytics and future quality improvement.
- If Twilio is replaced by another channel later, only the adapter/controller needs to change; the RAG service can remain the same.

3.3 Module 3: RAG Answer Generation and Hallucination Control

3.3.1 Module Description

The RAG Answer Generation and Hallucination Control module is the intelligence layer of the application. It receives a normalized customer question, converts it into an embedding, retrieves the most relevant document chunks, builds a grounded prompt, calls the generation model, checks whether the answer is supported by the retrieved context, and returns the final answer with citations.

- **Main functionalities:** query embedding, semantic retrieval, top-k chunk selection, prompt building, answer generation, relevance threshold checking, citation formatting, and safe fallback handling.
- **Inputs:** customer question, session id, optional chat context, top-k retrieval value, and similarity threshold.
- **Outputs:** answer text, citation list, confidence/relevance score, status code, and fallback response if needed.
- **Dependencies:** embedding model, vector database, metadata store, Hugging Face generation model, prompt template, and response formatter.

3.3.2 Internal Structure

```
+-------------------------------------------------------------+
| RAGQueryService                                             |
+-------------------------------------------------------------+
| - query_embedder: EmbeddingService                          |
| - retriever: VectorRetriever                                |
| - prompt_builder: PromptBuilder                             |
| - generator: GenerationService                              |
| - grounding_checker: GroundingChecker                       |
| - citation_formatter: CitationFormatter                     |
+-------------------------------------------------------------+
| + answer_question(question, session_id) -> RAGResult        |
+-------------------------------------------------------------+

VectorRetriever       -> retrieves top-k semantically similar chunks
PromptBuilder         -> builds an instruction prompt using only retrieved context
GenerationService     -> calls BART/T5 or another selected model
GroundingChecker      -> blocks weak answers when similarity/context is insufficient
CitationFormatter     -> adds source document and page references
```

3.3.3 Pseudo-code

```
FUNCTION answer_question(question, session_id):
    query_vector = embedding_service.encode(question)
    retrieved_chunks = vector_store.search(query_vector, top_k=3)

    IF retrieved_chunks is empty:
        RETURN RAGResult(status="NO_CONTEXT", answer="Limited Information Available")

    best_score = retrieved_chunks[0].similarity_score
    IF best_score < SIMILARITY_THRESHOLD:
        RETURN RAGResult(status="NO_CONTEXT", answer="Limited Information Available")

    context_text = ""
    citations = []
    FOR each chunk IN retrieved_chunks:
        context_text += "Source: " + chunk.file_name + ", Page: " + chunk.page_number
        context_text += "\n" + chunk.text + "\n---\n"
        citations.append({
            "file_name": chunk.file_name,
            "page_number": chunk.page_number,
            "chunk_id": chunk.chunk_id
        })

    prompt = prompt_builder.create(
        instruction = "Answer only from the provided context. If the context is insufficient,
say Limited Information Available.",
        context = context_text,
        question = question
    )

    draft_answer = generation_model.generate(prompt, max_tokens=180)

    IF grounding_checker.is_supported(draft_answer, retrieved_chunks) == False:
        RETURN RAGResult(status="NO_CONTEXT", answer="Limited Information Available")

    final_answer = citation_formatter.append(draft_answer, citations)
    RETURN RAGResult(status="OK", answer=final_answer, citations=citations)
```

3.3.4 Design Notes

- The module retrieves the top three relevant chunks because the SRS specifies a top-3 retrieval approach for answer generation.
- A similarity threshold prevents weak matches from being used as evidence. This supports the hallucination-control requirement.
- The prompt explicitly instructs the generation model to answer only from the retrieved context.
- Citations are generated from metadata saved during document ingestion, not from the model itself. This makes citations more reliable.
- A simple keyword-mapping layer from the original proposal can still be kept as an optional fallback for very common business triggers such as hours, pricing, or location. However, the main answer path remains RAG-based.

4. Design Evaluation

4.1 Modularity

The system is divided into manageable parts by separating interface handling, business logic, AI processing, and data storage. The dashboard handles administrator actions, the webhook receives external customer messages, the ingestion service builds the knowledge base, the RAG service produces answers, and repositories handle data access. This modular structure reduces coupling because each component has a clear responsibility and communicates with other components through defined inputs and outputs.

For example, the document ingestion service does not need to know how Twilio works. Similarly, the WhatsApp webhook does not need to understand the internal details of PDF extraction or vector indexing. This separation makes the system easier to debug and easier to explain during evaluation.

4.2 Maintainability and Extensibility

The design supports maintainability because common responsibilities are grouped into dedicated services. If the team later changes the embedding model, most changes will remain inside the EmbeddingService. If FAISS is replaced by ChromaDB, the VectorStoreRepository can be updated without changing the webhook or dashboard. If a new frontend is needed, it can call the same backend API.

The design is also extensible. Future versions can add email support, a human handover module, role-based administrator access, more analytics, multi-business workspaces, or a stronger production deployment without rewriting the core RAG logic. The use of environment variables also improves maintainability because API keys can be changed without editing code files.

4.3 Scalability

The proposed architecture can scale gradually. In the MVP, all services may run on one machine for simplicity. As usage grows, the webhook service can be deployed separately from the document ingestion worker, and the vector database can run as a dedicated service. Since webhook requests are stateless apart from session storage, multiple backend instances can be added behind a load balancer in a production environment.

Document ingestion can also be moved to a background queue because embedding large documents may take more time than answering a normal WhatsApp message. The RAG query path can be optimized by keeping vector search fast, limiting top-k context size, caching repeated questions, and using a model serving endpoint for Hugging Face models. These improvements help the system handle more users, more documents, and more services over time.

4.4 Trade-offs

Design Decision	Benefit	Trade-off / Compromise
Layered microservice-oriented design	Clear separation, easier future scaling, better testing.	More design complexity than a single script.
RAG instead of pure keyword mapping	Better natural language understanding and document-grounded answers.	Requires embeddings, vector storage, and careful latency control.
Hugging Face models	Open-source and flexible model choices.	May need hardware optimization or external inference for faster responses.
FAISS/ChromaDB vector store	Fast semantic search and suitable for MVP.	Requires metadata management to support citations properly.
ngrok for development	Easy local webhook testing.	Not suitable as a permanent production networking solution.
Top-3 retrieval	Keeps prompts compact and supports faster answers.	May miss context if documents are large or badly chunked.

4.5 Alternative Designs Considered

Several alternatives were considered before choosing the proposed architecture:

- **Simple monolithic keyword bot:** This would be easier to build, but it would not satisfy the SRS goal of natural language understanding and document-grounded answers. It would also fail when users ask questions in different wording.
- **Pure LLM chatbot without RAG:** This could generate fluent answers, but it may hallucinate and would not reliably answer from the uploaded business documents. It would also make source citations difficult.
- **Full production microservices with Kubernetes:** This would support strong scalability, but it is too complex for a semester MVP and would increase setup time significantly.
- **Serverless webhook-only design:** This could reduce server management, but it may complicate local development, vector database integration, and model hosting.

The selected design is therefore a balanced choice. It is structured enough to support the SRS requirements and future growth, but still simple enough to build and demonstrate as a semester project.

4.6 Security, Reliability, and Performance Considerations

- **Security:** Twilio credentials, Hugging Face tokens, and other secrets should be stored in environment variables. Webhook endpoints should validate requests where possible.
- **Reliability:** All failed uploads, failed API calls, and model errors should be logged. If the AI pipeline fails, the system should return a polite fallback message instead of crashing.
- **Performance:** Semantic search should remain fast by using vector indexing. The system should keep prompts short by retrieving only the most relevant chunks. Response latency should be tracked in chat logs.
- **Data quality:** The knowledge base should reject unsupported or unreadable files so that poor data does not affect answer accuracy.

5. Conclusion

The AI-Powered WhatsApp Business Agent is designed as a layered, microservice-oriented system that connects WhatsApp communication with a RAG-based AI knowledge engine. The design separates the administrator dashboard, webhook handling, document ingestion, RAG answer generation, vector storage, and external integrations. This separation makes the system modular, maintainable, extensible, and suitable for gradual scaling.

The detailed design focuses on three core modules: document ingestion, WhatsApp message orchestration, and RAG answer generation. Together, these modules support the main project goal: allowing a business to upload its own documents and automatically answer customer questions through WhatsApp with reliable, document-grounded responses and citations. The selected architecture provides a practical balance between simplicity for an academic MVP and flexibility for future SaaS-level development.